Software Impacts 22 (2024) 100716



Contents lists available at ScienceDirect

Software Impacts

journal homepage: www.journals.elsevier.com/software-impacts



Original software publication

Transformer-based deep learning architecture for time series forecasting



G.H. Harish Nayak [a,b,c], Md Wasi Alam [a], G. Avinash [d], Rajeev Ranjan Kumar [a,*], Mrinmoy Ray [a,*], Samir Barman [e], K.N. Singh [a], B. Samuel Naik [f], Nurnabi Meherul Alam [g], Prasenjit Pal [h], Santosha Rathod [i], Jaiprakash Bisen [a]

[a] ICAR-Indian Agricultural Statistics Research Institute, New Delhi, 110012, India
[b] The Graduate School, ICAR-Indian Agricultural Research Institute, New Delhi, 110012, India
[c] University of Agricultural Sciences, Dharwad, Karnataka, 580005, India
[d] ICMR-National Institute of Occupational Health, Ahmedabad, Gujarat, 380016, India
[e] ICAR-Indian Grassland and Fodder Research Institute, Jhansi, Uttar Pradesh 284003, India
[f] Institute of Agricultural Sciences, Banaras Hindu University, Uttar Pradesh, 221005, India
[g] ICAR-Central Research Institute for Jute and Allied Fibers, Kolkata, West Bengal 700120, India
[h] College of Fisheries, Central Agricultural University, Tripura 799210, India
[i] ICAR-Indian Institute of Rice Research, Hyderabad, Telangana 500030, India

ARTICLE INFO

Keywords:
Deep learning
Time series forecasting
Transformer

ABSTRACT

Time series forecasting faces challenges due to the non-stationarity, nonlinearity, and chaotic nature of the data. Traditional deep learning models like RNNs, LSTMs, and GRUs process data sequentially but are inefficient for long sequences. To overcome the limitations of these models, we proposed a transformer-based deep learning architecture utilizing an attention mechanism for parallel processing, enhancing prediction accuracy and efficiency. This paper presents user-friendly code for the implementation of the proposed transformer-based deep learning architecture utilizing an attention mechanism for parallel processing.

Code metadata

Current code version	V1
Permanent link to code/repository used for this code version	https://github.com/SoftwareImpacts/SIMPAC-2024-137
Code Ocean compute capsule	https://codeocean.com/capsule/8888820/tree/v1
Legal Code License	MIT
Code versioning system used	git
Software code languages, tools, and services used	Python 3
Compilation requirements, operating environments & dependencies	Code can be executed in Anaconda (Jupyter Notebook) or Google Colab
If available, link to developer documentation/manual	N/A
Support email for questions	rrk.uasd@gmail.com

1. Introduction

Time series forecasting, crucial across various domains, has seen increased research interest due to its potential to provide timely and accurate information. The inherent complexities of time series data, *viz.,* non-stationarity, nonlinearity and chaos, pose significant forecasting challenges [1]. Traditional parametric nonlinear models often fall short due to stringent assumptions, leading to the rise of machine learning approaches, which, while data-driven, lack automatic feature extraction capabilities. This limitation has boosted the popularity of deep learning

The code (and data) in this article has been certified as Reproducible by Code Ocean: (https://codeocean.com/). More information on the Reproducibility Badge Initiative is available at https://www.elsevier.com/physical-sciences-and-engineering/computer-science/journals.

* Corresponding authors.
E-mail addresses: rrk.uasd@gmail.com (R.R. Kumar), mrinmoy4848@gmail.com (M. Ray).

https://doi.org/10.1016/j.simpa.2024.100716
Received 30 September 2024; Received in revised form 25 October 2024; Accepted 29 October 2024

2665-9638/© 2024 Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

models. However, conventional deep learning architectures like RNNs, LSTMs, and GRUs process input sequentially, which can be time-consuming for long sequences [2]. To overcome these issues, we propose a transformer-based deep learning architecture that leverages an attention mechanism to parallelize sequential data processing. Originally designed for machine translation, the Transformer model's self-attention mechanisms effectively assess the importance of different input elements, enhancing prediction accuracy and learning speed [3]. This makes the Transformer model highly efficient for time series forecasting, offering a robust solution to the limitations of traditional models.

In this study, the model is implemented using Python 3.9 and the Keras and TensorFlow libraries [4]. These libraries provide a high-level interface for building and training deep learning models. The study is being run on a system with an AMD Ryzen 7 5700U processor and 8 GB of RAM. This configuration should be sufficient for training and evaluating the model. Python is invaluable for running deep learning models due to its extensive libraries and frameworks like TensorFlow, PyTorch, and Keras, which simplify model development and deployment. Its readable syntax and strong community support make it accessible and resource-rich, while its integration capabilities ensure versatility across various stages of the deep learning workflow. Python's robust tools for data manipulation and visualization, platform independence, and support for parallelism and GPU acceleration further enhance its utility, making it a preferred choice for developing and deploying deep learning models efficiently.

2. Step-by-step methodology

Below is a detailed breakdown of the proposed algorithm for time series forecasting:

1. **Input:** Time series data $S = \{s, s_2, \ldots, s_T\}$.
2. **Output:** Predicted values $\hat{S} = \{\hat{s}_{T+1}, \hat{s}_{T+2}, \ldots, \hat{s}_{T+h}\}$ for h future steps.
3. **Pre-processing Stage:**
   - **Normalization:** Transform the data to $S_{norm} = \frac{S - min(S)}{max(S) - min(S)}$.
   - **Missing Data** (if present): Address missing values using imputation techniques.
4. **Data Preparation:**
   - Data Splitting: Partition S_{norm} into a training set (S_{train}) and a test set (S_{test}).
   - Format Transformation:
     - Inputs: Create sequences $X = \{x_{t-1}, x_{t-2}, \ldots, x_{t-w}\}$, where x_i is a sliding window of values from S_{train}.
     - Outputs: Generate sequences $Y = \left\{x'_t, x'_{t+1}, \ldots, x'_{t+k}\right\}$, representing future values.
5. **Model Hyperparameters:**
   - Layers count: L.
   - Model dimensions: d_{model}.
   - Feedforward network size: d_{ff}.
   - Attention heads: H.
   - MLP units: M.
   - Dropout rate: δ.
6. **Model Construction:**
   - Embedding Input: Generate embeddings $E(X)$.
   - Add Positional Encoding: Compute $PE(X) = E(X) + positional_encoding(X)$.
   - Transformer Encoder: Apply multiple encoder layers with each encoder layer incorporating multi-head attention and feed-forward sub-layers.
7. **Training Phase:**
   - Train the model: Use the pairs (X, Y) from S_{train}.
   - Objective function: Use Mean Squared Error (MSE) to measure the loss.
8. **Forecasting Phase:**
   - Predict using S_{test}: Obtain predictions $\hat{S} = Model(S_{test})$.
9. **Post-Processing**:
   - Denormalization: Convert back to original scale $\hat{S}_{original} = \hat{S}_{norm} \times (max(S) - min(S)) + min(S)$.
10. **Evaluation Metrics:**
    - Calculate RMSE: $RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(\hat{S} - S_{true}\right)^2}$.
    - Calculate MAPE: $MAPE = \frac{1}{n}\sum_{i=1}^{n}\left|\frac{S_{true} - \hat{S}}{S_{true}}\right| \times 100$.

A diagram illustrating the proposed algorithm is depicted in Fig. 2.

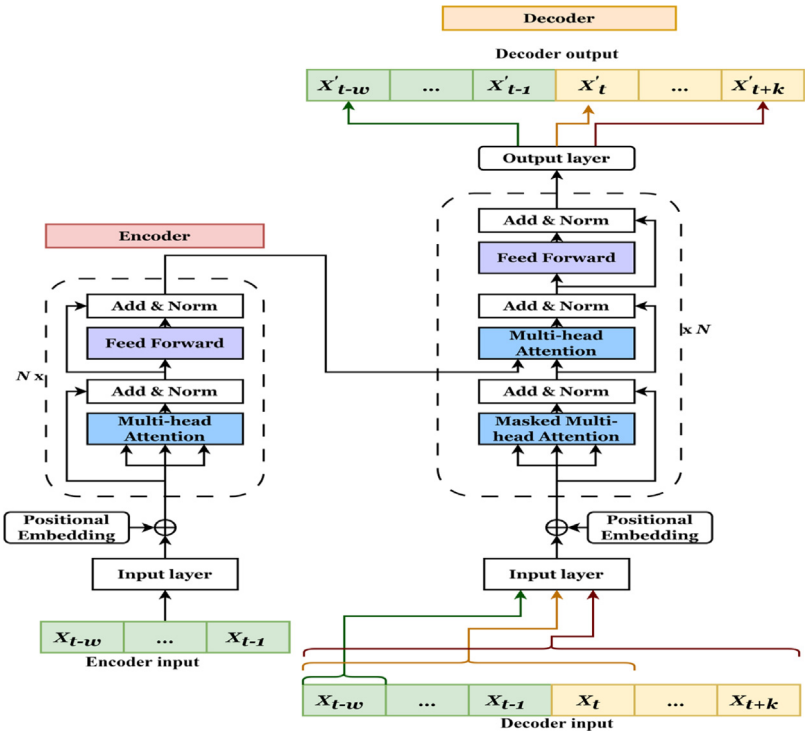


Fig. 1. Transformer architecture.

3. Related work

This transformer-based deep learning model (Fig. 1) has proven to be applicable across a spectrum of time series forecasting tasks. Numerous studies have leveraged transformer models, attesting to their adaptability and efficacy. For example, Bai and Tahmasebi [5] delved into groundwater contamination analysis using transformer-based deep learning architectures. Shi et al. [6] substantiated the superior performance of the Transformer model across various deep learning domains. Sasal et al. [7] introduced the W-Transformer, a wavelet-based framework tailored to capture extensive dependencies in non-stationary series. Castangia et al. [8] demonstrated the effectiveness of the Transformer model in flood forecasting, emphasizing its advantages over recurrent neural networks. Nayak et al. [3] proposed a transformer-encoder model specifically for rainfall prediction.

4. Impact on related fields

Transformer models have revolutionized time series forecasting across various domains by addressing key limitations of traditional methods. Their self-attention mechanism allows for capturing long-range dependencies and complex temporal patterns without the need for recurrent structures, making them particularly effective for multivariate and long-horizon forecasts. In finance, Transformers have improved stock price prediction and risk assessment [9]. For energy and climate applications, Transformers have boosted renewable energy generation forecasts and climate trend predictions [10]. The model's ability to handle multiple input variables simultaneously has proven valuable in scenarios with diverse influencing factors [11]. Additionally, Transformers' scalability and parallelization capabilities have enabled more efficient processing of large-scale time series data, leading to faster and more accurate forecasts in real-time applications. Their adaptability to transfer learning has also allowed for improved performance in domains with limited historical data.

A recent implementation of this technique for modeling monthly rainfall in India compared the performance of the Transformer model with other well-established deep learning models [3]. The results showed superior performance of the Transformer model on both training and testing sets, proving its superiority over other models. The proposed Transformer model significantly improves the accuracy of monthly rainfall forecasting by capturing long-range dependencies and integrating spatial–temporal data. It reduces forecasting errors, making it effective for detecting seasonal anomalies like droughts or floods. This model is scalable, making it suitable for large datasets and supporting applications in agriculture, water management, and climate adaptation.

5. Future developments

In the future, this model can be hybridized with other models, such as a CNN-Transformer hybrid, to obtain improved results. Additionally, a Transformer model capable of incorporating exogenous variables could also be developed. The performance of deep learning models heavily depends on hyperparameters, and optimizing these requires metaheuristic algorithms, an area that remains under-researched. Therefore, there is significant potential for developing optimized models to achieve improved forecasting results.

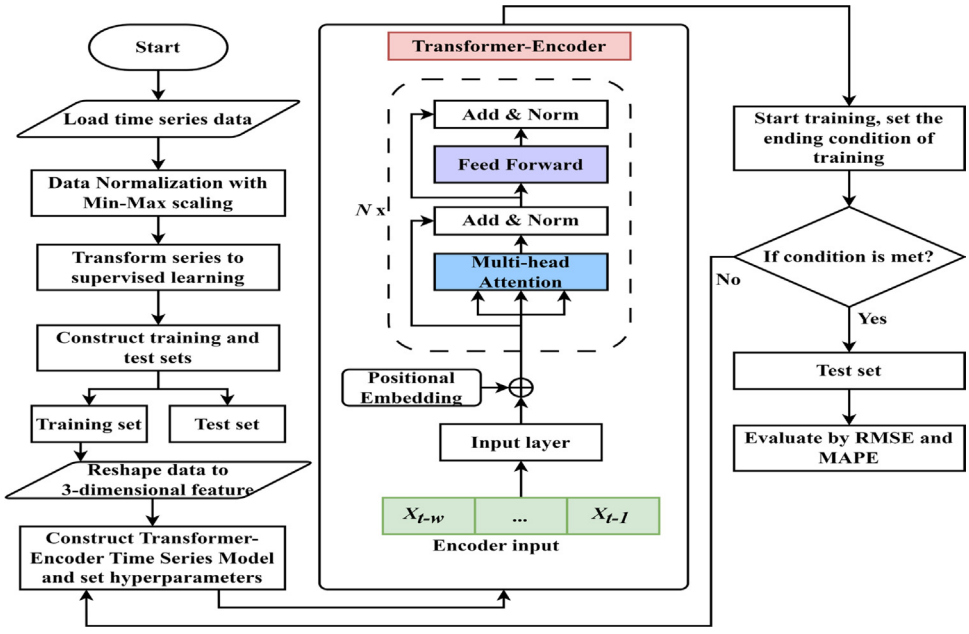


Fig. 2. A flowchart illustrating the proposed algorithm.

6. Publication:

Scholarly publications enabled by this technique:

Nayak, G. H., Alam, W., Singh, K. N., Avinash, G., Ray, M., & Kumar, R. R. (2024). Modelling monthly rainfall of India through transformer-based deep learning architecture. *Modeling Earth Systems and Environment*, 1–18. https://doi.org/10.1007/s40808-023-01944-7.

7. Packages details

Package name	Version	Website link
Pandas	1.4.2	pandas - Python Data Analysis Library (pydata.org) [12]
numpy	1.21.5	NumPy -- [13]
matplotlib	3.5.1	Matplotlib --- Visualization with Python [14]
sciPy	1.4.1	SciPy -- [15]
statsmodels	0.13.2	statsmodels 0.14.1 [16]
Scikit-learn	1.0.2	scikit-learn: machine learning in Python --- scikit-learn 1.4.2 documentation [17]
keras	2.8.0	Keras: Deep Learning for humans [18]
TensorFlow	2.8.0	TensorFlow [19]

8. Implementing a transformer-based deep learning model for time series forecasting using python

```python
import pandas as pd
import numpy as np
from tensorflow import keras
from tensorflow.keras import layers
from sklearn import metrics
import matplotlib.pyplot as plt
from keras.utils import plot_model

# Load and inspect data
df = pd.read_csv("Monthly_Rainfall.csv", sep=',', index_col=False)
print("Starting file:")
print(df.head(10))  # Print the first 10 rows of the dataset
print("Ending file:")
print(df.tail(10))  # Print the last 10 rows of the dataset
# Calculate the split index and split the data
split_index = int(len(df) * 0.9)  # Use 90% of the data for training and 10% for testing
df_train = df[:split_index]
df_test = df[split_index:]
rainfall_train = df_train['rainfall'].tolist()  # Extract rainfall data for training
rainfall_test = df_test['rainfall'].tolist()   # Extract rainfall data for testing

print("Training set has {} observations.".format(len(rainfall_train)))
print("Test set has {} observations.".format(len(rainfall_test)))
# Function to create sequences for training the model
def to_sequences(seq_size, obs):
    x, y = [], []
    for i in range(len(obs) - seq_size):
        window = obs[i:(i + seq_size)]  # Create a window of specified sequence size
        after_window = obs[i + seq_size]  # The target value after the window
        window = [[x] for x in window]  # Reshape window for model input
        x.append(window)
        y.append(after_window)
    return np.array(x), np.array(y)

SEQUENCE_SIZE = 10  # Length of the sequence for the model
x_train, y_train = to_sequences(SEQUENCE_SIZE, rainfall_train)
x_test, y_test = to_sequences(SEQUENCE_SIZE, rainfall_test)

print("Shape of training set: {}".format(x_train.shape))
print("Shape of training set: {}".format(y_train.shape))
print("Shape of test set: {}".format(x_test.shape))
print("Shape of test set: {}".format(y_test.shape))
# Define the transformer encoder
def transformer_encoder(inputs, head_size, num_heads, ff_dim, dropout=0):
    x = layers.LayerNormalization(epsilon=1e-6)(inputs)
    x = layers.MultiHeadAttention(key_dim=head_size, num_heads=num_heads, dropout=dropout)(x, x)
    x = layers.Dropout(dropout)(x)
    res = x + inputs

    x = layers.LayerNormalization(epsilon=1e-6)(res)
    x = layers.Conv1D(filters=ff_dim, kernel_size=1, activation="relu")(x)
    x = layers.Dropout(dropout)(x)
    x = layers.Conv1D(filters=inputs.shape[-1], kernel_size=1)(x)
    return x + res
# Build the transformer model
def build_model(input_shape, head_size, num_heads, ff_dim, num_transformer_blocks, mlp_units, dropout=0, mlp_dropout=0):
    inputs = keras.Input(shape=input_shape)
    x = inputs
    for _ in range(num_transformer_blocks):
        x = transformer_encoder(x, head_size, num_heads, ff_dim, dropout)
    x = layers.GlobalAveragePooling1D(data_format="channels_first")(x)
    for dim in mlp_units:
        x = layers.Dense(dim, activation="relu")(x)
        x = layers.Dropout(mlp_dropout)(x)
    outputs = layers.Dense(1)(x)
    return keras.Model(inputs, outputs)

input_shape = x_train.shape[1:]  # Input shape for the model
model = build_model(input_shape, head_size=512, num_heads=4, ff_dim=4, num_transformer_blocks=4, mlp_units=[128],
                    mlp_dropout=0.4, dropout=0.25)
model.compile(loss="mean_squared_error", optimizer=keras.optimizers.Adam(learning_rate=1e-4))
callbacks = [keras.callbacks.EarlyStopping(patience=10, restore_best_weights=True)]  # Early stopping callback

# Train the model
history = model.fit(x_train, y_train, validation_split=0.2, epochs=300, batch_size=64, callbacks=callbacks)
# Evaluate the model on the test set
model.evaluate(x_test, y_test, verbose=1)
pred = model.predict(x_test)
score = np.sqrt(metrics.mean_squared_error(pred, y_test))
mape = metrics.mean_absolute_percentage_error(pred, y_test) * 100
print("Score (RMSE): {}".format(score))
print("Score (MAPE): {}".format(mape))

# Visualize the model architecture
plot_model(model, to_file='model_plot.png', show_shapes=True, show_layer_names=True)
# Plot Actual vs Predicted values
plt.figure(dpi=600)
plt.plot(y_test, label='Actual', color='blue', linewidth=2)
plt.plot(pred, label='Predicted', color='red', linewidth=2, linestyle='--')
plt.legend(fontsize='medium', loc='upper left')
plt.xlabel('Time (Monthly)', fontsize=16, fontweight='bold')
plt.ylabel('Rainfall (cm)', fontsize=16, fontweight='bold')
plt.xticks(fontsize=14, fontweight='bold')
plt.yticks(fontsize=14, fontweight='bold')
plt.savefig('Transformer_predicted_vs_actual.jpeg', format='jpeg', dpi=600)
plt.show()

# Plot Training and Validation Loss
plt.figure(dpi=600)
plt.plot(history.history['loss'], label='Training Loss', color='g', linewidth=2)
plt.plot(history.history['val_loss'], label='Validation Loss', color='b', linewidth=2)
plt.legend(fontsize='medium', loc='upper right')
plt.xlabel('Epochs', fontsize=16, fontweight='bold')
plt.ylabel('Loss', fontsize=16, fontweight='bold')
plt.xticks(fontsize=14, fontweight='bold')
plt.yticks(fontsize=14, fontweight='bold')
plt.savefig('loss_plot.jpeg', format='jpeg', dpi=600)
plt.show()
```

CRediT authorship contribution statement

G.H. Harish Nayak: Writing – original draft, Software, Methodology, Formal analysis, Data curation, Conceptualization. **Md Wasi Alam:** Supervision, Software, Conceptualization. **G. Avinash:** Formal analysis, Data curation, Conceptualization. **Rajeev Ranjan Kumar:** Writing –

review & editing, Validation, Supervision, Software, Conceptualization. **Mrinmoy Ray:** Writing – original draft, Software, Methodology, Formal analysis, Data curation, Conceptualization. **Samir Barman:** Methodology, Investigation, Formal analysis, Conceptualization. **K.N. Singh:** Validation, Supervision, Conceptualization. **B. Samuel Naik:** Methodology, Data curation. **Nurnabi Meherul Alam:** Validation, Software. **Prasenjit Pal:** Validation, Software, Formal analysis. **Santosha Rathod:** Software, Methodology. **Jaiprakash Bisen:** Writing – review & editing, Validation.

Funding

No Funding.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors extend their gratitude to ICAR-IASRI and The Graduate School, ICAR-IARI, New Delhi, for generously providing software and other essential resources.

References

[1] G. Avinash, V. Ramasubramanian, M. Ray, R.K. Paul, S. Godara, G.H. Nayak, R.R. Kumar, B. Manjunatha, S. Dahiya, M.A. Iquebal, Hidden Markov guided deep learning models for forecasting highly volatile agricultural commodity prices, Appl. Soft Comput. 158 (2024) 111557.
[2] Z. Li, X. Zhang, Z. Dong, TSF-transformer: a time series forecasting model for exhaust gas emission using transformer, Appl. Intell. 53 (2023) 17211–17225, http://dx.doi.org/10.1007/s10489-022-04326-1.
[3] G.H. Nayak, W. Alam, K.N. Singh, G. Avinash, M. Ray, R.R. Kumar, Modelling monthly rainfall of India through transformer-based deep learning architecture, Model. Earth Syst. Environ. (2024) 1–18.
[4] TensorFlow in Python package system- PyPI, Python Software Foundation, URL https://www.tensorflow.org.
[5] T. Bai, P. Tahmasebi, Characterization of groundwater contamination: A transformer-based deep learning model, Adv. Water Resour. 164 (2022) 104217, http://dx.doi.org/10.1016/j.advwatres.2022.104217.
[6] J. Shi, M. Jain, G. Narasimhan, Time Series Forecasting (TSF) Using Various Deep Learning Models, 2022, http://dx.doi.org/10.48550/arXiv.2204.11115.
[7] L. Sasal, T. Chakraborty, A. Hadid, W-transformers: A wavelet-based transformer framework for univariate time series forecasting, in: 2022 21st IEEE International Conference on Machine Learning and Applications, ICMLA, 2022.
[8] M. Castangia, L.M.M. Grajales, A. Aliberti, C. Rossi, A. Macii, E. Macii, E. Patti, Transformer neural networks for interpretable flood forecasting, Environ. Model. Softw. 160 (2023) 105581.
[9] C. Wang, Y. Chen, S. Zhang, Q. Zhang, Stock market index prediction using deep transformer model, Expert Syst. Appl. 208 (2022) 118128.
[10] P. Agrawal, H.O. Bansal, A.R. Gautam, O.P. Mahela, B. Khan, Transformer-based time series prediction of the maximum power point for solar photovoltaic cells, Energy Sci. Eng. 10 (9) (2022) 3397–3410.
[11] G.H. Nayak, M.W. Alam, K.N. Singh, G. Avinash, R.R. Kumar, M. Ray, C.K. Deb, Exogenous variable driven deep learning models for improved price forecasting of TOP crops in India, Sci. Rep. 14 (1) (2024) 17203.
[12] W. McKinney, Pandas: A foundational Python library for data analysis and statistics, Python high perform, Sci. Comput. 14 (2011) 1–9.
[13] C.R. Harris, K.J. Millman, S.J. Van Der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N.J. Smith, R. Kern, Array programming with NumPy, Nature 585 (7825) (2020) 357–362.
[14] P. Barrett, J. Hunter, J.T. Miller, J.C. Hsu, P. Greenfield, Matplotlib–A portable Python plotting package, in: Astronom-Ical Data Analysis Software and Systems XIV, vol. 347, 2005, p. 91.
[15] P. Virtanen, R. Gommers, T.E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S.J. van der Walt, M. Brett, J. Wilson, K.J. Millman, N. Mayorov, A.R.J. Nelson, E. Jones, R. Kern, E. Larson, C.J. Carey, İ. Polat, Y. Feng, E.W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E.A. Quintero, C.R. Harris, A.M. Archibald, A.H. Ribeiro, F. Pedregosa, P. van Mulbregt, SciPy 1.0 contributors, SciPy 1.0: Fundamental algorithms for scientific computing in Python, Nat. Methods 17 (2020) 261–272, http://dx.doi.org/10.1038/s41592-019-0686-2.
[16] S. Seabold, J. Perktold, Statsmodels: Econometric and statis-tical modeling with Python, in: Proc. 9th Python Sci. Conf, 2010, pp. 92–96, http://dx.doi.org/10.25080/majora-92bf1922-011.
[17] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, Scikit-learn: Machine learning in Python, J. Mach. Learn. Res. 12 (2011) 2825–2830.
[18] keras in Python package system- PyPI, Python Software Foundation, URL https://keras.io.
[19] F.J.J. Joseph, S. Nonsiri, A. Monsakul, Keras and TensorFlow: A hands-on experience, in: K.B. Prakash, R. Kannan, S. Alexander, G.R. Kanagachidambaresan (Eds.), Advanced Deep Learning for Engineers and Scientists, in: EAI/Springer Innovations in Communication and Computing, Springer, Cham, 2021, http://dx.doi.org/10.1007/978-3-030-66519-7_4.